

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Вычислительная математика»
ТЕМА: РЕШЕНИЕ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ
УРАВНЕНИЙ

Студент гр. 4341

Сердюков Ф.С.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучение метода решения систем линейных уравнений методом Гаусса и написание программы, реализующей данный метод.

Задание.

В ходе выполнения работы студенты должны найти решение системы линейных уравнений с n неизвестными, заданной матрицей коэффициентов и вектором свободных членов, методом Гаусса. Выполнение работы состоит из следующих этапов:

1) с помощью преподавателя определить систему уравнений, которую нужно решить;

2) для решения системы уравнений разработать программу на языке C, использующую подпрограмму-функцию GAUSS из файла GAUSS.CPP. Данная функция имеет следующие параметры:

- a - матрица коэффициентов системы уравнений размера $n \times n$, тип double ;
- b - вектор свободных членов размера $n \times 1$, тип double ;
- x - выходной вектор результата решения размера $n \times 1$, тип double ;
- n - размер системы (матрицы a и вектора свободных членов), тип int .

В разрабатываемой программе должна быть описана константа $n_{\max}$, равная максимальным размерам используемых матриц и векторов. Функция GAUSS в качестве значения типа int возвращает:

- а) 0 - в случае нормального завершения процесса вычисления;
- б) 1 - в случае вырожденности матрицы a ;
- в) 2 - если $n > n_{\max}$;
- г) 3 - если $n < 0$.

Провести вычисления с использованием разработанной программы и исследовать обусловленность задачи с использованием пакета Matlab, при этом для определения числа обусловленности матрицы A рекомендуется использовать функцию $\text{cond}(A)$ [14]. Кроме того, для проверки получаемых результатов можно провести вычисления с помощью пакетов Matlab и Derive [15].

Разработка программы

Программа, реализующая метод Гаусса решения систем линейных алгебраических уравнений, была написана на языке C++.

Реализация метода Гаусса с частичным выбором главного элемента

Была написана функция `solveGauss(A, b, n, pivotType)`, которая координирует решение системы линейных алгебраических уравнений (СЛАУ). Функция принимает матрицу коэффициентов A (размером $N \times N$) и вектор свободных членов b (длиной N) по значению, что автоматически создает их копии и предотвращает изменение исходных данных. Также передается размерность системы n и тип выбора ведущего элемента `pivotType`. Для частичного выбора главного элемента вызываются функции `gaussForwardPartial(A, b, n, permutations)` и `gaussBackwardPartial(A, b, n)`.

Вычисления производятся с использованием типа `double` для обеспечения высокой точности работы с плавающей точкой.

Прямой ход метода Гаусса с частичным выбором

В функции `gaussForwardPartial` на каждом шаге k (от 0 до $n-1$) выполняется частичный выбор главного элемента:

Из элементов k -го столбца, начиная с k -й строки и ниже, ищется максимальный по модулю элемент.

Фиксируется индекс строки `maxRow`, содержащей этот элемент.

Если `maxRow` не равен текущей строке `k`, строки матрицы `A` и соответствующие элементы вектора `b` меняются местами с помощью функции `std::swap`. При этом увеличивается счетчик перестановок `permutations`.

Этот шаг необходим для повышения численной устойчивости метода — он позволяет избежать деления на малые числа, способные вызвать лавинный рост ошибок округления.

Далее для всех строк `i` от `k+1` до `n-1` вычисляется множитель:

`factor = A[i][k] / pivot` (где `pivot` — это ведущий элемент `A[k][k]`)

После этого из `i`-й строки вычитается `k`-я строка, умноженная на `factor`. Одновременно аналогичное преобразование применяется к элементам вектора `b`, а исключенный элемент `A[i][k]` зануляется. В результате к концу прямого хода матрица `A` приобретает верхнетреугольный вид.

Обратный ход

Обратный ход реализован в функции `gaussBackwardPartial`. Он начинается с последнего уравнения и идет в обратном направлении от `n-1` до `0`. Решение вычисляется по формуле, где сумма произведений известных коэффициентов на уже найденные значения неизвестных рассчитывается в цикле:

`sum += A[i][j] * x[j]` (для `j` от `i+1` до `n-1`)

После расчета суммы вычисляется значение переменной:

`x[i] = (b[i] - sum) / A[i][i]`

Функция возвращает вектор `x`, содержащий найденное решение системы.

Для предотвращения деления на ноль предусмотрена проверка: если максимальный по модулю элемент на текущем шаге оказывается меньше $1e-12$, матрица считается вырожденной (или близкой к вырожденной). Функция выводит предупреждение «Матрица вырождена!» и прекращает работу, возвращая значение `false`.

Реализация метода Гаусса с полным выбором главного элемента

Дополнительно были написаны функции `gaussForwardFull` и `gaussBackwardFull`, реализующие метод Гаусса с полным выбором ведущего элемента. В отличие от частичного выбора, здесь максимальный по модулю элемент ищется во всей оставшейся подматрице $A[i:][j:]$ (где индексы строк и столбцов больше или равны текущему шагу k), а не только в одном столбце.

Особенности реализации:

Поиск максимального элемента выполняется с помощью двух вложенных циклов по строкам и столбцам от k до $n-1$, определяющих индексы `maxRow` и `maxCol`.

Помимо перестановки строк (как в частичном выборе), выполняется перестановка столбцов матрицы A , что эквивалентно перенумерации неизвестных.

Для отслеживания перестановок столбцов используется вектор `pivotCol`, изначально заполненный номерами столбцов от 0 до $n-1$. При каждой перестановке столбцов соответствующие элементы в `pivotCol` меняются местами.

По завершении обратного хода в функции `gaussBackwardFull` рассчитывается промежуточное решение `x_temp` для системы с

переставленными столбцами. Для восстановления исходного порядка неизвестных выполняется обратная перестановка: $x[pivotCol[i]] = x_temp[i]$.

Полный выбор обеспечивает высокую численную устойчивость алгоритма, но требует больше времени на поиск максимального значения в двумерном массиве на каждом шаге.

Функция вычисления определителя

Для оценки характеристик матрицы реализована функция `determinant(A, n, pivotType)`. Она выполняет прямой ход метода Гаусса (частичный или полный в зависимости от `pivotType`) и вычисляет определитель как произведение диагональных элементов полученной треугольной матрицы. При этом учитывается количество выполненных перестановок строк (и столбцов): если их число нечетное, знак определителя меняется на противоположный.

Проверка точности и замер времени

В отличие от использования сторонних библиотек, в данной программе оценка результатов встроена непосредственно в основную логику функции `main`:

Для замера времени выполнения выбранного метода решения используется библиотека `<chrono>`. Фиксируются моменты времени непосредственно перед вызовом `solveGauss` и сразу после него. Разность переводится в миллисекунды.

Программа сравнивает полученное решение x с известным истинным вектором решений `x_true` (сгенерированным на этапе создания системы) и выводит абсолютную ошибку для каждого элемента, а также максимальное отклонение (`maxError`).

Дополнительно на исходных (не измененных прямым ходом) данных вычисляется вектор невязки. Величина невязки рассчитывается как максимум абсолютной величины разности между Ax и b : $\text{residual} = \max |Ax - b|$. По значению невязки программа автоматически выводит качественную оценку точности («высокая точность», «хорошая точность» или «невысокая точность»).

Генерация тестовых матриц

В программе предусмотрены две функции для генерации систем уравнений:

`generateRandomMatrix(n, A, b, x_true)` — заполняет матрицу A случайными вещественными числами в диапазоне от -10.0 до 10.0 с использованием генератора `std::mt19937`. Вектор истинных решений x_true также заполняется случайными значениями, а вектор правой части b вычисляется как произведение матрицы A на x_true . Это гарантирует совместность системы.

`generateHilbertMatrix(n, A, b, x_true)` — генерирует матрицу Гильберта, элементы которой вычисляются по формуле $A[i][j] = 1.0 / (i + j + 1)$. Данные матрицы являются классическим примером плохо обусловленных систем, что позволяет наглядно исследовать устойчивость алгоритмов. Истинное решение x_true задается вектором из единиц, после чего вычисляется правая часть b .

Исследование

Программа позволяет проводить интерактивное исследование для различных размерностей n , задаваемых пользователем:

Пользователь может выбрать размерность системы (при $n > 1000$ выводится предупреждение о возможных затратах времени).

Доступен выбор типа генерируемой матрицы (случайная или Гильберта).

Доступен выбор метода (частичный или полный выбор главного элемента).

Для систем малых порядков ($n \leq 10$) предусмотрен наглядный вывод расширенной матрицы системы на экран. Для больших систем выводятся первые 5 строк.

Разработанный программный код представлен в Приложении А.

Цель работы.

Разработать программу для решения систем линейных алгебраических уравнений методом простой итерации (Якоби) для диагонально преобладающих матриц и исследовать сходимость метода.

Задание.

Разработать программу для решения систем линейных алгебраических уравнений (СЛАУ) методом простой итерации (Якоби) для матриц с диагональным преобладанием. Программа должна генерировать диагонально преобладающую матрицу заданного размера, выполнять итерационный процесс до достижения заданной точности, выводить результаты итераций и сравнивать полученное решение с истинным.

Разработка программы решения СЛАУ методом простой итерации (Якоби)

Программа реализует метод простой итерации (Якоби) для решения систем линейных алгебраических уравнений (СЛАУ) с диагональным преобладанием. Она состоит из нескольких логических частей: генерации диагонально преобладающей матрицы, проверки достаточных условий сходимости,

вычисления нормы матрицы перехода, самого итерационного процесса и оценки погрешности.

Генерация тестовой матрицы с диагональным преобладанием

Функция `generateDiagonallyDominantMatrix(n, A, b, x_true)` создает матрицу, для которой математически гарантируется сходимость метода Якоби. Принцип генерации следующий:

Сначала с помощью генератора случайных чисел `std::mt19937` недиагональные элементы строки заполняются значениями в диапазоне от -5.0 до 5.0.

Вычисляется сумма модулей всех недиагональных элементов текущей строки (`offDiagSum`).

Диагональный элемент вычисляется как сумма `offDiagSum` и дополнительного случайного значения в диапазоне от 2.0 до 10.0. Это гарантирует строгое диагональное преобладание: $|a_{ii}| > \sum |a_{ij}|$ (для всех j не равных i) для каждой строки системы.

Для верификации метода случайным образом генерируется истинное решение `x_true`. Если случайная величина по модулю оказывается меньше 0.1, элемент принудительно заменяется на 1.0 во избежание генерации тривиального (околонулевого) решения. Вектор правой части `b` рассчитывается как произведение матрицы `A` на вектор `x_true`, что делает систему заведомо совместной.

Проверка условий сходимости

Функция `checkDiagonalDominance(A, n)` проверяет выполнение достаточного условия сходимости метода Якоби. Для каждой строки матрицы вычисляется модуль диагонального элемента и сравнивается с суммой модулей внедиагональных элементов.

Если для всех строк диагональный элемент строго больше суммы, то матрица обладает диагональным преобладанием. Если хотя бы в одной строке обнаруживается равенство (с точностью до $1e-10$) или нарушение условия, функция возвращает `false`, предупреждая, что сходимость метода не гарантирована.

Вычисление нормы матрицы итераций

Вспомогательная функция `iterationMatrixNorm(A, n)` оценивает скорость сходимости метода Якоби, вычисляя бесконечную норму матрицы итераций по формуле:

$$q = \max_i (\text{сумма по всем } j \neq i \text{ от } |a_{ij}| / |a_{ii}|)$$

Данная норма напрямую влияет на скорость сходимости процесса: чем меньше значение q , тем быстрее приближенное решение стремится к точному. При $q < 1$ сходимость метода гарантируется.

Итерационный процесс метода Якоби

Функция `solveJacobi` осуществляет итерационное нахождение вектора решений. Перед началом итераций выполняются проверки:

На отсутствие нулей на главной диагонали (с точностью до $1e-12$).

На наличие диагонального преобладания.

На соответствие нормы матрицы перехода условию сходимости ($q < 1$).

Итерационный процесс начинается с нулевого начального приближения. На каждом шаге новое приближение x_new рассчитывается на основе значений вектора x с предыдущего шага по формуле метода Якоби:

$$x_new[i] = (b[i] - \text{сумма по всем } j \neq i \text{ от } A[i][j] * x[j]) / A[i][i]$$

После вычисления всех элементов x_new рассчитывается максимальное абсолютное изменение вектора неизвестных за шаг (норма разности):

$$\text{maxDiff} = \max |x_new[i] - x[i]|$$

Затем вектор текущего приближения x обновляется значениями из x_new . Процесс итераций продолжается до тех пор, пока величина maxDiff не станет меньше заданной пользователем точности epsilon .

Для наглядности хода вычислений в консоль выводятся подробные сведения: полностью выводятся первые 10 итераций, а затем каждая 100-я итерация. Также обязательно отображается последняя завершающая итерация. Если размерность системы $n \leq 5$, то вектор приближения x печатается полностью, в противном случае выводятся только первые три компоненты. С помощью библиотеки `chrono` фиксируется время работы итерационного цикла в миллисекундах.

Оценка погрешности и невязки решения

После выхода из итерационного цикла в функции `main` оценивается качество полученного решения:

Вычисляется максимальное отклонение полученных неизвестных от сгенерированного истинного решения: $\text{maxError} = \max |x[i] - x_true[i]|$.

Вычисляется невязка по формуле $\text{residual} = \max |A * x - b|$, показывающая, насколько точно полученный вектор решает исходное уравнение.

Если значение невязки оказывается строго меньше ϵ , то выводится заключение, что решение найдено с заданной точностью.

Интерактивное исследование

Интерфейс программы в функции `main` позволяет пользователю:

Ввести желаемый размер матрицы n .

Ввести требуемую точность вычислений ϵ .

Оценить структуру матрицы (если $n \leq 10$, то выводится вся система коэффициентов, если $n > 10$ — выводятся первые 5 строк).

Программа устойчиво обрабатывает различные ограничения, предотвращает выполнение на заведомо расходящихся системах и информирует пользователя о промежуточных характеристиках процесса сходимости.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab2.1.cpp

```
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <random>
#include <chrono>
#include <fstream>

using namespace std;

// Генерация случайной матрицы
void generateRandomMatrix(int n, vector<vector<double>>& A,
vector<double>& b, vector<double>& x_true) {
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<double> dist(-10.0, 10.0);

    A.assign(n, vector<double>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            A[i][j] = dist(gen);
        }
    }

    // Истинное решение (случайное)
    x_true.assign(n, 0.0);
    for (int i = 0; i < n; i++) {
        x_true[i] = dist(gen);
    }

    // Правая часть b = A * x_true
    b.assign(n, 0.0);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            b[i] += A[i][j] * x_true[j];
        }
    }
}

// Генерация матрицы Гилберта
void generateHilbertMatrix(int n, vector<vector<double>>& A,
vector<double>& b, vector<double>& x_true) {
    A.assign(n, vector<double>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            A[i][j] = 1.0 / (i + j + 1);
        }
    }
}
```

```

// Истинное решение - все единицы
x_true.assign(n, 1.0);

// Правая часть
b.assign(n, 0.0);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        b[i] += A[i][j] * x_true[j];
    }
}

// Вывод системы
void printSystem(const vector<vector<double>>& A, const
vector<double>& b, int n) {
    cout << "\nСистема уравнений:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << setw(10) << setprecision(4) << A[i][j] << " ";
        }
        cout << " | " << setw(10) << setprecision(4) << b[i] << endl;
    }
    cout << endl;
}

// Вывод решения
void printSolution(const vector<double>& x, const vector<double>&
x_true, int n) {
    cout << "\nРешение:\n";
    double maxError = 0.0;
    for (int i = 0; i < n; i++) {
        double error = fabs(x[i] - x_true[i]);
        maxError = max(maxError, error);
        cout << "x" << i + 1 << " = " << setw(12) << setprecision(8)
<< x[i];
        cout << " (истинное: " << setw(12) << setprecision(8)
<< x_true[i] << ")";
        cout << " ошибка: " << error << endl;
    }
    cout << "\nМаксимальная ошибка: " << maxError << endl;
}

// Прямой ход с ЧАСТИЧНЫМ выбором (только по
столбцу)
bool gaussForwardPartial(vector<vector<double>>& A, vector<double>&
b, int n, int& permutations) {
    cout << "\nИспользуем ЧАСТИЧНЫЙ выбор ведуще
го элемента (по столбцу)\n";

    permutations = 0;

    for (int k = 0; k < n; k++) {
        // Поиск максимального элемента в стол
бце k
        int maxRow = k;

```

```

double maxVal = fabs(A[k][k]);
for (int i = k + 1; i < n; i++) {
    if (fabs(A[i][k]) > maxVal) {
        maxVal = fabs(A[i][k]);
        maxRow = i;
    }
}

if (maxVal < 1e-12) {
    cout << "Матрица вырождена!" << endl;
    return false;
}

// Перестановка строк (если нужно)
if (maxRow != k) {
    swap(A[k], A[maxRow]);
    swap(b[k], b[maxRow]);
    permutations++;
    cout << " Шаг " << k + 1 << ": переставили с
т р о к и " << k + 1 << " и " << maxRow + 1 << endl;
}

// Исключение
double pivot = A[k][k];
for (int i = k + 1; i < n; i++) {
    double factor = A[i][k] / pivot;
    for (int j = k + 1; j < n; j++) {
        A[i][j] -= factor * A[k][j];
    }
    b[i] -= factor * b[k];
    A[i][k] = 0.0;
}
}
return true;
}

// Прямой ход с ПОЛНЫМ выбором (по всей матри
це) - с сохранением перестановок
bool gaussForwardFull(vector<vector<double>>& A, vector<double>& b,
int n,
vector<int>& pivotCol, int& permutations) {
    cout << "\nИспользуем ПОЛНЫЙ выбор ведущего
э л е м е н т а (п о в с е й м а т р и ц е)\n";

    vector<int> pivotRow(n);
    for (int i = 0; i < n; i++) {
        pivotRow[i] = i;
        pivotCol[i] = i;
    }

    permutations = 0;

    for (int k = 0; k < n; k++) {
        // Поиск максимального элемента во все
й о с т а в ш е й с я п о д м а т р и ц е
        double maxVal = 0.0;

```

```

int maxRow = k, maxCol = k;
for (int i = k; i < n; i++) {
    for (int j = k; j < n; j++) {
        if (fabs(A[i][j]) > maxVal) {
            maxVal = fabs(A[i][j]);
            maxRow = i;
            maxCol = j;
        }
    }
}

if (maxVal < 1e-12) {
    cout << "Матрица вырождена!" << endl;
    return false;
}

// Перестановка строк
if (maxRow != k) {
    swap(A[k], A[maxRow]);
    swap(b[k], b[maxRow]);
    swap(pivotRow[k], pivotRow[maxRow]);
    permutations++;
    cout << " Шаг " << k + 1 << ": переставили с
т р о к и " << k + 1 << " и " << maxRow + 1 << endl;
}

// Перестановка столбцов
if (maxCol != k) {
    for (int i = 0; i < n; i++) {
        swap(A[i][k], A[i][maxCol]);
    }
    swap(pivotCol[k], pivotCol[maxCol]);
    permutations++;
    cout << " Шаг " << k + 1 << ": переставили с
т о л б ц ы " << k + 1 << " и " << maxCol + 1 << endl;
}

// Исключение
double pivot = A[k][k];
for (int i = k + 1; i < n; i++) {
    double factor = A[i][k] / pivot;
    for (int j = k + 1; j < n; j++) {
        A[i][j] -= factor * A[k][j];
    }
    b[i] -= factor * b[k];
    A[i][k] = 0.0;
}

return true;
}

// Обратный ход с восстановлением порядка п е
р е м е н н ы х (для полного выбора)
vector<double> gaussBackwardFull(const vector<vector<double>>& A,
const vector<double>& b,

```



```

int n, const vector<int>& pivotCol)
{
    vector<double> x_temp(n);

    // Обратный ход (решаем систему с перестав
ленными столбцами)
    for (int i = n - 1; i >= 0; i--) {
        double sum = 0.0;
        for (int j = i + 1; j < n; j++) {
            sum += A[i][j] * x_temp[j];
        }
        x_temp[i] = (b[i] - sum) / A[i][i];
    }

    // Восстанавливаем исходный порядок пере
менных
    vector<double> x(n);
    for (int i = 0; i < n; i++) {
        x[pivotCol[i]] = x_temp[i];
    }

    return x;
}

// Обратный ход (для частичного выбора - без
восстановления)
vector<double> gaussBackwardPartial(const vector<vector<double>>& A,
const vector<double>& b, int n) {
    vector<double> x(n);
    for (int i = n - 1; i >= 0; i--) {
        double sum = 0.0;
        for (int j = i + 1; j < n; j++) {
            sum += A[i][j] * x[j];
        }
        x[i] = (b[i] - sum) / A[i][i];
    }
    return x;
}

// Вычисление определителя
double determinant(vector<vector<double>> A, int n, int pivotType) {
    vector<double> b(n, 0.0);
    int permutations = 0;

    if (pivotType == 2) {
        vector<int> pivotCol(n);
        if (!gaussForwardFull(A, b, n, pivotCol, permutations)) {
            return 0.0;
        }
    } else {
        if (!gaussForwardPartial(A, b, n, permutations)) {
            return 0.0;
        }
    }

    double det = 1.0;
    for (int i = 0; i < n; i++) {

```

```

        det *= A[i][i];
    }

    // У ч и т ы в а е м  з н а к  о т  п е р е с т а н о в о к
    if (permutations % 2 == 1) {
        det = -det;
    }

    return det;
}

// Р е ш е н и е  С Л А У  м е т о д о м  Г а у с с а  с  в ы б о р о м  м е т
о д а
vector<double> solveGauss(vector<vector<double>> A, vector<double> b,
int n, int pivotType) {
    cout << "\n--- Р е ш е н и е  м е т о д о м  Г а у с с а  ---\n";

    auto start = chrono::high_resolution_clock::now();

    vector<double> x;
    int permutations;

    if (pivotType == 2) {
        // П о л н ы й  в ы б о р
        vector<int> pivotCol(n);
        if (!gaussForwardFull(A, b, n, pivotCol, permutations)) {
            return {};
        }
        x = gaussBackwardFull(A, b, n, pivotCol);
    } else {
        // Ч а с т и ч н ы й  в ы б о р
        if (!gaussForwardPartial(A, b, n, permutations)) {
            return {};
        }
        x = gaussBackwardPartial(A, b, n);
    }

    auto end = chrono::high_resolution_clock::now();
    auto total_time =
chrono::duration_cast<chrono::milliseconds>(end - start).count();

    cout << "\nВ р е м я  р е ш е н и я: " << total_time << " м с " <<
endl;

    return x;
}

int main() {
    cout << "===== " << endl;
    cout << "М Е Т О Д  Г А У С С А " << endl;
    cout << "===== " << endl;

    int n;
    cout << "В в е д и т е  р а з м е р  м а т р и ц ы  n : ";
    cin >> n;

```

```

    if (n <= 0) {
        cout << "Некорректный размер!" << endl;
        return 1;
    }

    if (n > 1000) {
        cout << "Предупреждение: размер больше 1000
может привести к долгому выполнению" << endl;
        cout << "Продолжить? (1 - да, 0 - нет): ";
        int confirm;
        cin >> confirm;
        if (confirm != 1) {
            return 0;
        }
    }

    vector<vector<double>> A;
    vector<double> b, x_true;

    cout << "\nВыберите тип матрицы:" << endl;
    cout << "1 - Случайная матрица" << endl;
    cout << "2 - Матрица Гилберта (плохо обусловл
енная)" << endl;
    int matrixType;
    cin >> matrixType;

    if (matrixType == 2) {
        generateHilbertMatrix(n, A, b, x_true);
        cout << "\nСгенерирована матрица Гилберта
" << endl;
    } else {
        generateRandomMatrix(n, A, b, x_true);
        cout << "\nСгенерирована случайная матриц
а" << endl;
    }

    cout << "\nВыберите метод выбора ведущего эл
емента:" << endl;
    cout << "1 - Частичный выбор (по столбцу, быст
рее)" << endl;
    cout << "2 - Полный выбор (по всей матрице, то
чнее)" << endl;
    int pivotType;
    cin >> pivotType;

    // Сохраняем копии для проверки невязки
    vector<vector<double>> A_original = A;
    vector<double> b_original = b;

    // Вывод системы (для маленьких матриц)
    if (n <= 10) {
        printSystem(A, b, n);
    } else {
        cout << "\nРазмер матрицы: " << n << "x" << n <<
endl;

```

```

        cout << "Первые 5 строк:" << endl;
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                cout << setw(10) << setprecision(4) << A[i][j] << "
";
            }
            cout << " ... | " << b[i] << endl;
        }
        cout << "..." << endl;
    }

    // Вычисление определителя
    double det = determinant(A, n, pivotType);
    cout << "\nОпределитель матрицы: " << det << endl;

    if (fabs(det) < 1e-10) {
        cout << "Внимание: определитель близок к
нулю, матрица плохо обусловлена" << endl;
    }

    // Решение
    vector<double> x = solveGauss(A, b, n, pivotType);

    if (!x.empty()) {
        printSolution(x, x_true, n);

        // Проверка невязки на оригинальной ма
т р и ц е
        double residual = 0.0;
        for (int i = 0; i < n; i++) {
            double sum = 0.0;
            for (int j = 0; j < n; j++) {
                sum += A_original[i][j] * x[j];
            }
            residual = max(residual, fabs(sum - b_original[i]));
        }
        cout << "\nНевязка (max |Ax - b|): " << scientific <<
residual << fixed << endl;

        // Оценка качества решения
        if (residual < 1e-10) {
            cout << "Решение найдено с высокой точ
ностью" << endl;
        } else if (residual < 1e-6) {
            cout << "Решение найдено с хорошей точ
ностью" << endl;
        } else {
            cout << "Точность решения невысокая, в
озможно, матрица плохо обусловлена" << endl;
        }
    }

    return 0;
}

```

Название файла: lab2.2.cpp

```
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <random>
#include <chrono>

using namespace std;

// Генерация диагонально преобладающей матрицы
void generateDiagonallyDominantMatrix(int n, vector<vector<double>>& A,
vector<double>& x_true) {
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<double> dist(-5.0, 5.0);
    uniform_real_distribution<double> diagDist(2.0, 10.0);

    A.assign(n, vector<double>(n, 0.0));

    for (int i = 0; i < n; i++) {
        double offDiagSum = 0.0;

        // Генерируем недиагональные элементы
        for (int j = 0; j < n; j++) {
            if (i != j) {
                A[i][j] = dist(gen);
                offDiagSum += fabs(A[i][j]);
            }
        }

        // Диагональный элемент больше суммы остальных
        A[i][i] = offDiagSum + diagDist(gen);
    }

    // Истинное решение (случайное)
    x_true.assign(n, 0.0);
    for (int i = 0; i < n; i++) {
        x_true[i] = dist(gen);
        if (fabs(x_true[i]) < 0.1) {
            x_true[i] = 1.0;
        }
    }

    // Правая часть b = A * x_true
    b.assign(n, 0.0);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            b[i] += A[i][j] * x_true[j];
        }
    }
}
```

```

    }

    // Проверка диагонального преобладания
    bool checkDiagonalDominance(const vector<vector<double>>& A, int n)
    {
        cout << "\nПроверка диагонального преобладания:" << endl;
        bool hasDominance = true;

        for (int i = 0; i < n; i++) {
            double diag = fabs(A[i][i]);
            double sum = 0.0;
            for (int j = 0; j < n; j++) {
                if (i != j) {
                    sum += fabs(A[i][j]);
                }
            }

            cout << "Строка " << i + 1 << ": |" << diag << "| vs "
            << sum;

            if (diag > sum) {
                cout << " - преобладание есть" << endl;
            } else if (fabs(diag - sum) < 1e-10) {
                cout << " - равенство (сходимость не гарантирована)" << endl;
                hasDominance = false;
            } else {
                cout << " - ПРЕОБЛАДАНИЯ НЕТ" << endl;
                hasDominance = false;
            }
        }

        return hasDominance;
    }

    // Вычисление нормы матрицы итераций
    double iterationMatrixNorm(const vector<vector<double>>& A, int n) {
        double norm = 0.0;
        for (int i = 0; i < n; i++) {
            double rowSum = 0.0;
            for (int j = 0; j < n; j++) {
                if (i != j) {
                    rowSum += fabs(A[i][j]) / fabs(A[i][i]);
                }
            }
            norm = max(norm, rowSum);
        }
        return norm;
    }

    // Вывод системы
    void printSystem(const vector<vector<double>>& A, const vector<double>& b, int n) {
        cout << "\nСистема уравнений:" << endl;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {

```

```

        cout << setw(10) << setprecision(4) << A[i][j] << " ";
    }
    cout << " | " << setw(10) << setprecision(4) << b[i] << endl;
}
cout << endl;
}

// Вывод решения
void printSolution(const vector<double>& x, const vector<double>&
x_true, int n, int iterations) {
    cout << "\nРешение (найдено за " << iterations << " и
т е р а ц и й):" << endl;
    double maxError = 0.0;
    for (int i = 0; i < n; i++) {
        double error = fabs(x[i] - x_true[i]);
        maxError = max(maxError, error);
        cout << "x" << i + 1 << " = " << setw(12) << setprecision(8)
<< x[i];
        cout << " (истинное: " << setw(12) << setprecision(8)
<< x_true[i] << ")";
        cout << " ошибка: " << error << endl;
    }
    cout << "\nМаксимальная ошибка: " << maxError << endl;
}

// Метод Якоби (простая итерация)
vector<double> solveJacobi(const vector<vector<double>>& A, const
vector<double>& b,
                        const vector<double>& x_true, int n,
                        double epsilon, int& iterationsDone) {

    // Проверка диагональных элементов
    for (int i = 0; i < n; i++) {
        if (fabs(A[i][i]) < 1e-12) {
            cout << "Ошибка: нулевой диагональный э
лемент в строке " << i + 1 << endl;
            return {};
        }
    }

    // Проверка диагонального преобладания
    if (!checkDiagonalDominance(A, n)) {
        cout << "\nОШИБКА: Метод Якоби требует диа
гонального преобладания!" << endl;
        return {};
    }

    // Вычисление нормы матрицы итераций
    double normM = iterationMatrixNorm(A, n);
    cout << "\nНорма матрицы итераций: " << normM <<
endl;

    if (normM >= 1.0) {
        cout << "ОШИБКА: Норма матрицы итераций >=
1, метод не сойдется!" << endl;
    }
}

```

```

        return {};
    } else {
        cout << "Сходимость гарантирована (норма
< 1)" << endl;
    }

    // Начальное приближение - нулевой вектор
    vector<double> x(n, 0.0);
    vector<double> x_new(n);

    cout << "\nИтерационный процесс:" << endl;
    cout << "-----
--" << endl;

    auto start = chrono::high_resolution_clock::now();

    int iter = 0;
    double maxDiff = 0.0;

    while (true) {
        iter++;

        // Одна итерация метода Якоби
        for (int i = 0; i < n; i++) {
            double sum = 0.0;
            for (int j = 0; j < n; j++) {
                if (j != i) {
                    sum += A[i][j] * x[j];
                }
            }
            x_new[i] = (b[i] - sum) / A[i][i];
        }

        // Вычисление максимального изменения
        maxDiff = 0.0;
        for (int i = 0; i < n; i++) {
            double diff = fabs(x_new[i] - x[i]);
            if (diff > maxDiff) {
                maxDiff = diff;
            }
        }

        // Обновление решения
        x = x_new;

        // Вывод промежуточных результатов
        if (iter <= 10 || iter % 100 == 0 || maxDiff < epsilon) {
            cout << "Итерация " << setw(5) << iter << ": изм
енение = " << scientific << setw(12) << maxDiff;
            if (n <= 5) {
                cout << fixed << "  x = (";
                for (int i = 0; i < n; i++) {
                    cout << setw(10) << setprecision(6) << x[i];
                    if (i < n - 1) cout << ", ";
                }
                cout << ")";
            }
        }
    }
}

```



```

        } else {
            cout << fixed << "    x[0] = " << setw(10) <<
setprecision(6) << x[0];
            cout << ", x[1] = " << setw(10) << setprecision(6)
<< x[1];
            cout << ", x[2] = " << setw(10) << setprecision(6)
<< x[2] << ", ...";
        }
        cout << endl;
    }

    // Проверка сходимости
    if (maxDiff < epsilon) {
        cout << "-----"
-----" << endl;
        cout << "С ОШЛОСЬ за " << iter << " и т е р а ц и й !"
<< endl;
        break;
    }
}

    auto end = chrono::high_resolution_clock::now();
    auto elapsed = chrono::duration_cast<chrono::milliseconds>(end -
start).count();

    cout << "В р е м я  в ы п о л н е н и я: " << elapsed << " м с " <<
endl;

    iterationsDone = iter;
    return x;
}

int main() {
    cout << "===== " << endl;
    cout << "МЕТОД ПРОСТОЙ ИТЕРАЦИИ (ЯКОБИ)" << endl;
    cout << "===== " << endl;
    cout << "\nВНИМАНИЕ: Метод работает только д
ля матриц" << endl;
    cout << "с диагональным преобладанием!" << endl;

    int n;
    cout << "\nВведите размер матрицы n: ";
    cin >> n;

    if (n <= 0) {
        cout << "Некорректный размер!" << endl;
        return 1;
    }

    vector<vector<double>> A;
    vector<double> b, x_true;

    // Генерируем диагонально преобладающую м
атрицу
    generateDiagonallyDominantMatrix(n, A, b, x_true);

```

```

        cout << "\nСгенерирована диагонально преобл
адающая матрица" << endl;

// Параметры метода
double epsilon;

cout << "\nВведите точность (например, 1e-8): ";
cin >> epsilon;

// Вывод системы (для маленьких матриц)
if (n <= 10) {
    printSystem(A, b, n);
} else {
    cout << "\nРазмер матрицы: " << n << "x" << n <<
endl;

    cout << "Первые 5 строк:" << endl;
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            cout << setw(10) << setprecision(4) << A[i][j] << "
";

        }
        cout << " ... | " << b[i] << endl;
    }
    cout << "..." << endl;
}

// Решение
int iterationsDone;
vector<double> x = solveJacobi(A, b, x_true, n, epsilon,
iterationsDone);

if (!x.empty()) {
    printSolution(x, x_true, n, iterationsDone);

    // Проверка невязки
    double residual = 0.0;
    for (int i = 0; i < n; i++) {
        double sum = 0.0;
        for (int j = 0; j < n; j++) {
            sum += A[i][j] * x[j];
        }
        residual = max(residual, fabs(sum - b[i]));
    }
    cout << "\nНевязка (max |Ax - b|): " << scientific <<
residual << fixed << endl;

    // Оценка качества
    if (residual < epsilon) {
        cout << "Решение найдено с заданной то
чностью" << endl;
    } else {
        cout << "Решение найдено, но точность н
иже заданной" << endl;
    }
}
}

```

```
    return 0;  
}
```